BORDERLESS
<CODING>

Rust for Web3 development

Part 01 · Introduction

Solana · Web3 mental model

Rust & Anchor · Solana Playground

Minimal program in Rust

About me

César Patiño

Senior Web3 & full-stack developer

+10 years of experience
+300 projects

Rust · Solana
Solidity · Hardhat
Python · Node.js · React · TypeScript

Instructor @ Borderless Coding

What you will learn

1. What is the **Solana** blockchain
2. The **Web3** mental model
3. The **Rust** and **Anchor** basics
4. How to use the **Solana Playground**
5. How to create a **minimal program in Rust**

What is the Solana blockchain

What is the Solana blockchain?

Solana is a blockchain that is designed to be fast, cheap, and scalable.

- **Layer-1 network**
- **Low gas fees**:
  0.000005 SOL per transaction
  (~ USD 0.0003-0.001)
- **Programs**: smart contracts
- **SOL**: the native currency of Solana

Solana network diagram

%%{init: {'flowchart': {'nodeSpacing': 40, 'rankSpacing': 60}, 'themeVariables': {'fontSize': '26px'}}}%% flowchart TB CW[Clients / wallets] -->|"signed transactions"| VC[Validator cluster] VC -->|"blocks + consensus"| BL[Shared ledger] BL --> RT[SVM runtime] RT --> PR[Programs] RT --> AC[(Accounts)] PR -.->|"instructions"| AC

Technologies

Blockchain: Solana

Programming Language: Rust

Framework: Anchor

Client Languages & Tests: JavaScript or TypeScript

Why this ecosystem?

- **Performance & throughput**
- **Strong tooling**: Playground, Anchor, wallets, explorers
- **Rust for on-chain logic**
- **Active developer community**

New Solana addresses by year

xychart-beta title "New addresses / year (approx., millions)" x-axis [2020, 2021, 2022, 2023, 2024, 2025] y-axis "Millions" 0 --> 120 bar [2, 14, 24, 58, 105, 98]

Source: The Block — monthly new addresses summed per calendar year (millions, rounded). 2020 from mainnet launch (Mar).

The Web3 mental model

Web3 (Solana)	Web2 analogy
Transaction	**Request**: succeeds or fails
Program	**Stateless service** or shared library (logic)
Account	Entry: **row** or **doc**, persists **bytes**
Wallet / signer	**Logged-in user** + permissions
Cluster (devnet / mainnet)	**Environment**: staging vs production

The Rust and Anchor basics

Rust: what you need to know

Rust is a programming language that is designed to be fast, safe, and reliable.

- **Low level**: no garbage collection, no runtime, no virtual machine
- **Memory-safe**: no null pointers, no undefined behavior
- **Performance**: fast, low-level, and memory-efficient

> *You do **not** need to be a Rust expert to follow along*

Anchor framework

Anchor is a framework that makes it easier to write Rust programs for the Solana blockchain.

Piece	Role
IDL + macros	Typed accounts & instructions
`#[program]`	Your instruction handlers
`#[derive(Accounts)]`	Accounts each instruction needs (Part 2)
vs raw SDK	Less boilerplate · safer defaults

Web2: classic backend

- Implicit **session / environment**
- State lives mainly in **one database** (PostgreSQL, MongoDB, etc.)
- Centralized server (API, database, etc.)

flowchart LR U[Client / browser] -->|HTTP| R[Route / API] R -->|Response| U R -->|Call| S[Service / logic] S -->|Query| D[(Central database)] D -->|Query result| S S -->|Service result| R

Web3: blockchain flow

Stateless program; you pass **accounts** every call — no hidden server memory.

flowchart LR W[Wallet / client] -->|" Signed transaction "| RT[Runtime] RT -->|Invoke| P[Program / logic] P -->|Read/Write| A1[(Account)] P -->|Read/Write| A2[(Account)] A1 -->|Account data| P A2 -->|Account data| P P -->|Program result| RT RT -->|Tx result / logs| W

Web3 key rules on Solana

- **Program** = stateless logic
- **Account** = durable state
- No separate database
- Every instruction declares:
  - **who signs**
  - **which accounts are writable**
  - and **arguments**

How to use the Solana Playground

Solana Playground

- Browser IDE: playground.solana.com
- **New project → Anchor** — same layout as local Anchor: `lib.rs`, client, tests
- Next slides: **full loop** (build → deploy → run) then **program + client + test** files

Playground — build & deploy

1. **Build** — compiles the on-chain program
2. **Deploy** — uploads the `.so` to Playground's cluster
3. **`declare_id!(...)`** in `lib.rs` must match that **Program ID**

- **If you change the id string**, you need to:
  - **Build** again
  - then **Deploy** again

Playground — run, client & tests

- Run test
- `client.ts`
- `anchor.test.ts`
- Test (no gas) → Deploy (gas) → Run (gas to write data)

How to create a minimal program in Rust

Hello, Solana! program variants

Native Rust: Hello, Solana! program

```rust
use solana_program::{
    account_info::AccountInfo,
    entrypoint,
    entrypoint::ProgramResult,
    msg,
    pubkey::Pubkey,
};

// In native Rust, `declare_id!` is OPTIONAL
// because `program_id` is passed into the
// entrypoint function.
// If you want to use a specific program ID,
// you can declare it with `declare_id!`.
entrypoint!(process_instruction);

fn process_instruction(
    _program_id: &Pubkey,
    _accounts: &[AccountInfo],
    _instruction_data: &[u8],
) -> ProgramResult {
    msg!("Hello, Solana!");
    Ok(())
}
```

Seahorse (Python): Hello, Solana! program

```python
from seahorse.prelude import *

declare_id("Fg6PaFpo...your_program_id...")

@instruction
def hello():
    print("Hello, Solana!")
```

Native Rust vs Anchor vs Seahorse

Approach	Pros	Cons
Native Rust (Solana, 2020)	Max control · minimal abstraction	More boilerplate · slower onboarding
Anchor (Rust, 2021)	Fast dev with safe defaults	Less low-level control
Seahorse (Python, 2022)	Python-friendly syntax · quick prototyping	Extra abstraction layer · smaller ecosystem

Playground: `lib.rs`

```rust
use anchor_lang::prelude::*;
use solana_program::msg;

declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFLSn");

#[program]
pub mod hello_anchor {
    use super::*;

    pub fn hello(_ctx: Context<Hello>) -> Result<()> {
        msg!("Hello, Solana!");
        Ok(())
    }
}

#[derive(Accounts)]
pub struct Hello {}
```

Link to code on GitHub

Playground: `client.ts`

```
const tx = await pg.program.methods.hello().rpc();

await pg.connection.confirmTransaction(tx, "confirmed");

const details = await pg.connection.getTransaction(tx, {
  commitment: "confirmed",
  maxSupportedTransactionVersion: 0,
});

console.log(details?.meta?.logMessages);
```

Link to code on GitHub

Playground: `anchor.test.ts`

```
describe("hello world", () => {
  it("calls hello and verifies the log string", async () => {
    const sig = await pg.program.methods.hello().rpc();
    if (!sig) throw new Error("no signature");

    const tx = await pg.connection.getTransaction(sig, {
      commitment: "confirmed",
      maxSupportedTransactionVersion: 0,
    });

    const logs = tx?.meta?.logMessages ?? [];
    const hasHelloLog = logs.some((line) => line.includes("Hello, Sola
    if (!hasHelloLog) throw new Error("expected 'Hello, Solana!' log")
  });
});
```

Link to code on GitHub

Exercise: edit the message

- Edit Rust (e.g change the message) → build → deploy → invoke → read logs
- Make sure the test still passes according to the new message
- Make sure the logs show the new message

Part 01 recap

- **Solana blockchain**
- **Web3**
  - transactions ≈ atomic requests
  - accounts ≈ rows
  - programs ≈ stateless logic
- **Rust + Anchor:** small set of syntax; Anchor adds structure and safety

Next (Part 2)

What you will learn in Part 2:

- Handlers in Programs
- Context and account bundles
- Account structs
- Common constraints

Read and write from/to Accounts

See you in Part 2!